ChatGPT

Design Document: Modular Prompt-to-Workflow Orchestration System

Goals

The primary goal of this Python package is to enable a **modular prompt-to-workflow system** that can translate high-level user prompts into structured, multi-step workflows. The system is initially focused on **data pipelines, content generation, and API-chaining workflows**, but it is designed with future extensibility for more complex scenarios (e.g. multi-agent simulations and deep research tasks). Key goals include:

- **Modularity:** Each workflow consists of discrete **agent** modules that perform specific functions (data fetch, transformation, LLM prompt, API call, etc.). These agents can be configured or swapped independently.
- **Workflow Orchestration:** Use **Tensor0** (the TensorZero orchestration layer) for **fast, concurrent execution** and built-in observability of steps. This ensures workflows execute efficiently and with transparent logging/tracing of each step [1] [2].
- **Structured Definitions:** Define workflows, agent roles, and functions using **Protobuf schemas**. This provides a clear, language-agnostic specification for workflows that can be versioned and validated.
- **Flexible Input Formats:** Allow workflows to be triggered by either **natural language prompts** (unstructured) or **structured JSON** inputs. This dual input design supports both user-friendly prompts and programmatic/automated triggers.
- **Deterministic Agent Spawning:** Workflows can spawn required agents **on-demand** in a deterministic way as per the workflow spec. Given the same specification and input, the same sequence of agent actions will occur (no nondeterministic agent creation at runtime).
- **Initial Scope & Future Extensibility:** Initially support triggered or UI-driven workflows (one-shot runs, not real-time interactive agents). The architecture should be extensible toward future **multi-agent systems** (multiple agents interacting or running concurrently) and **deep research workflows** (iterative, long-running analyses).

By meeting these goals, the system will empower users to turn high-level instructions into reliable automated workflows, with the agility of prompt-driven behavior and the rigor of structured pipeline execution.

Architecture Overview

High-level architecture of the prompt-to-workflow system.
The system follows a layered architecture separating **workflow definition**, **execution orchestration**, and **agent implementations**. At a high level, the flow is:

1. **User Input:** A user provides a prompt or structured JSON request to initiate a workflow. This could happen via a UI or an API trigger.
2. **Workflow Manager/Engine:** The core **Prompt-to-Workflow Engine** interprets the input and maps it to a specific workflow definition. If the input is natural language, the engine might

perform intent parsing or prompt matching to select the appropriate workflow. For JSON inputs, the workflow may be explicitly specified.

3. **Workflow Definition (Protobuf):** Each workflow is defined in a **Protobuf schema** that describes the sequence of steps, the roles of agents for each step, and how data flows between steps. The engine loads the appropriate `WorkflowSpec` from a registry or library of workflows [3].
4. **Tensor0 Orchestrator:** The engine leverages **Tensor0 (TensorZero)** as an orchestration layer to execute the workflow steps. Tensor0 coordinates the execution of prompt calls and tool/API calls, managing concurrency and capturing observability data (latency, traces, intermediate outputs) [4] [2]. This gives the workflow **fast execution** (sub-millisecond overhead per step in the orchestration layer [1]) and detailed logging/metrics out of the box.
5. **Agent Pool:** The system maintains an **Agent Pool**, which is a registry of available agent modules. Agents can be thought of as workers or tools that perform tasks – e.g., an LLM agent for text generation, a data processing agent, an API client agent, etc. The workflow engine requests agents from the pool as needed. Each workflow either defines its own agents (in the Protobuf spec) or draws from a global pool of shared agents.
6. **Agent Execution:** For each step in the workflow, the orchestrator either **reuses** an existing agent instance or **spawns** a new agent of the required type/role. The agent is provided with the inputs for that step (which could be user input or outputs from previous steps) and carries out its function. For example, an "API Caller" agent might execute an HTTP request, while an "LLM Generator" agent might call a language model with a prompt.
7. **Data Flow:** The system routes outputs from one step to the inputs of subsequent steps as defined in the workflow spec. The Tensor0 orchestrator and the engine ensure data dependencies are respected (sequential or parallel as specified). Complex workflows can include branching, parallel calls, or conditional logic (although initial version may focus on linear sequences).
8. **Output Aggregation:** After all steps complete, the engine collects the final results. The **workflow output** could be a structured result (JSON, file, etc.) or a natural language response, depending on the workflow. The output is then returned to the user or calling system, and relevant logs/metrics are recorded for observability.

This architecture cleanly separates **what** the workflow should do (defined by the Protobuf spec and agent roles) from **how** the workflow is executed (managed by the Tensor0 orchestration layer and engine). It ensures flexibility in updating workflows or agents without modifying core execution logic.

Key Components

Workflow Definition Schema (Protobuf)

Workflows and agents are formally specified using Protobuf schemas. This provides a rigorous contract for how workflows are composed, enabling validation and cross-language compatibility. By defining workflows as data (in `.proto` files), we ensure they can be stored, version-controlled, and even auto-generated into code in multiple languages if needed.

Workflow Schema: At minimum, a `WorkflowSpec` message defines the workflow name and a list of steps and agents. Each **step** can reference an agent (by role or ID) and defines the action that agent should perform (such as invoking a prompt or calling an API). Steps also declare their input sources (which could be outputs of previous steps or user inputs) and their output names for downstream use.

Agent Schema: An `AgentSpec` message defines an agent's identity, role, and capabilities. The role could be something like `"DataCleaner"`, `"ContentGenerator"`, or `"APICaller"`, and

capabilities might include what functions or tools the agent has. For LLM-powered agents, the spec might include a model name or prompt template.

Below is an **illustrative Protobuf schema** for the workflow system (simplified for clarity):

```
syntax = "proto3";

message WorkflowSpec {
  string name = 1;
  repeated Step steps = 2;
  repeated AgentSpec agents = 3;
}

message Step {
  string id = 1;
  string agent_id = 2;          // Reference to an AgentSpec by ID
  repeated string inputs =
3;  // Names of inputs (could map to previous step outputs or user prompt
fields)
  repeated string outputs = 4; // Names of outputs produced by this step
  oneof action {
    LLMCall llm_call = 5;
    APICall api_call = 6;
    // Future action types: ToolUse, DataTransform, etc.
  }
}

message LLMCall {
  string prompt_template = 1;  // e.g. "Summarize data: ${data}"
}

message APICall {
  string endpoint = 1;
  string method = 2;
  string body_template =
3;    // Template for request payload (could include placeholders from
inputs)
}

message AgentSpec {
  string id = 1;
  string role = 2;
  string model =
3;            // (optional) For LLM agents, the model or API to use
  repeated string functions = 4; // (optional) Tool or API function names
this agent can invoke
}
```

In this schema, a workflow lists all required agents and the sequence of steps. For example, a workflow might list an agent "A1" with role "DataFetcher", an agent "A2" with role "Summarizer", and then have

steps like: Step1 uses A1 to perform an `APICall`, Step2 uses A2 to perform an `LLMCall` using Step1's output. The use of Protobuf means these definitions are strongly typed and can evolve (new fields or step types can be added in the future without breaking older workflow specs).

Using a schema-based approach is inspired by the need for **structured orchestration** in LLM and tool workflows. Notably, TensorZero itself emphasizes **schemas for inputs/outputs** to maintain robustness [5]. By enforcing a schema, we ensure each prompt or action in the workflow adheres to an expected format, reducing errors and easing debugging.

Prompt-to-Workflow Engine

The **workflow engine** is the central brain that ties everything together. Its responsibilities include:

- **Input Interpretation:** Taking the user's input (prompt or JSON) and determining which workflow to run and how to map input data to the workflow's expected inputs. For natural language prompts, this might involve using a simple intent classifier or keyword matching to choose a workflow. For structured inputs, it might directly contain the workflow name or ID.
- **Loading Workflow Specs:** Fetching the corresponding `WorkflowSpec` (likely by name or ID) from a repository of compiled Protobuf definitions. This spec informs what steps and agents are needed.
- **Initializing Agents:** For each agent listed in the workflow spec, the engine either initializes a new agent instance or retrieves one from the **Agent Pool**. The agent pool might pre-instantiate some common agents (like an LLM interface) or maintain stateful agents that persist across runs (though initial implementation may keep agents stateless per run).
- **Invoking Orchestrator:** The engine translates the workflow steps into actual execution. It uses Tensor0's orchestration API to schedule each step's execution. Tensor0's role here is to manage the low-level details of running tasks (possibly in parallel where possible) and capturing their outputs and metrics.
- **Data Passing:** The engine makes sure outputs from steps are correctly passed as inputs to subsequent steps, according to the workflow spec's dataflow. This may involve temporarily storing outputs in a context that agents can access.
- **Error Handling & Observability:** If any step fails (e.g., an API call returns an error, or the LLM call times out), the engine can either retry, skip, or fail the workflow according to predefined policies. Thanks to Tensor0's observability features, each inference or API call can be logged with trace IDs and performance metrics [2]. The engine can augment these with higher-level workflow logs (e.g., step start/end, agent spawn events).
- **Output Construction:** Once all steps are complete, the engine collects the final outputs. If the workflow is meant to produce a single result (like a generated document or a numeric summary), that is formatted as needed (possibly by a final agent or by the engine itself) and returned. If multiple outputs, they might be packaged in a JSON response or saved to files, etc., as defined by the workflow.

The engine essentially acts as a **conductor**: reading the music (workflow spec) and cueing the musicians (agents via orchestrator) to play in the right sequence. It ensures the user's high-level prompt is fulfilled by the coordinated work of multiple agents.

Tensor0 Orchestration Layer

Tensor0 (TensorZero) serves as the high-performance backbone for executing steps. Rather than writing our own task scheduler, we integrate Tensor0 for its speed and robust features:

- *Fast Execution:* Tensor0 is designed for high-throughput, low-latency LLM and tool calls. It introduces minimal overhead (on the order of 1ms or less) even under heavy loads [1], which is crucial as workflows might chain several calls.
- *Multi-step Workflow Support:* Tensor0 natively supports multi-step LLM workflows through the concept of **episodes** [6]. We leverage this by treating each workflow run as an episode, where each step's LLM call or action is logged as part of the same session. This allows linking steps together in logs and possibly feeding outputs to later prompts with context.
- *Observability:* All calls made via Tensor0 can be captured with structured logs. For instance, every LLM prompt, parameters, and the resulting output can be stored, along with timing information [2]. The orchestration layer can also log tool invocations and their results. This aligns with our goal of **observable execution** – users and developers can inspect what happened in a run, see intermediate results, and diagnose issues or refine prompts.
- *Concurrency:* While the initial focus might be linear workflows, Tensor0 can handle parallel calls (for example, if two API calls in a workflow can run simultaneously, or if we scale to multi-agent scenarios where agents operate in parallel). The orchestrator can schedule these concurrently while the engine simply defines the dependency graph (DAG) of the workflow.
- *Integration:* Tensor0 provides a unified interface to various model providers and tools [7]. In our design, this means an agent that needs an LLM call doesn't directly call an external API; it goes through Tensor0 (which might route to OpenAI, Azure, local model, etc., depending on config). Similarly, tool usage might be abstracted via Tensor0's interface, which simplifies agent implementation.

By using Tensor0, we offload many complex aspects of workflow execution to a proven orchestration system. Our package will include a thin integration layer (e.g., a Python client that translates our workflow steps into Tensor0 API calls or jobs). This gives us reliability and performance out of the gate, and the ability to scale or incorporate new optimizations from the TensorZero project seamlessly (such as prompt caching, fallback models, A/B testing, etc., which TensorZero supports [8]).

Agents and Agent Pool

Agents are the modular units of functionality in the system. Each agent encapsulates a capability or role – for example, one agent might be responsible for calling a REST API, another for running an LLM prompt, another for performing a data transformation (like filtering or summarizing data).

Characteristics of the agent system design:

- **Standard Interface:** All agents implement a common interface, e.g., a Python class with a method like `execute(inputs) -> outputs`. This allows the orchestrator to invoke any agent in a uniform way. The agent reads the necessary fields from `inputs` (which could be a dict or Protobuf message matching a schema) and returns its result in a standardized format.
- **Types of Agents:** Initially, we foresee a few core agent types:
- *LLM Agent:* wraps an LLM model call. It might accept a prompt (possibly templated with input data) and produce a completion. Under the hood, it calls Tensor0's API to perform the inference.
- *API Agent:* performs external API calls. It would use input parameters or data to formulate an HTTP request (or database query, etc.), execute it, and return the response. It could use Tensor0's tool integration if available or a direct HTTP client.

- *Data Processing Agent:* runs arbitrary Python code or scripts for data manipulation – e.g., parsing JSON, cleaning text, combining results from multiple sources. This could be implemented using small Pandas or custom logic within the agent.
- *Aggregator/Decision Agent:* (possibly later) makes decisions on branching or looping by analyzing intermediate results (for now, workflows can be static, but in future an agent could decide the next step).
- **Agent Pool:** Rather than constructing each agent from scratch for every workflow invocation, the system can maintain a pool or registry of agent instances or classes:
- *Reusability:* If two workflows use the same kind of agent (say an `EmailSender` or a `DatabaseQuery` agent), they can share a single definition in the pool. The engine can instantiate a new one as needed or reuse a long-lived instance (especially if an agent holds state like an authenticated API session).
- *Modularity:* New agents can be added to the pool (by installing a plugin or package) without altering existing workflows. For example, if a new kind of model or service becomes available, one can write a new agent for it and then reference it in future workflow definitions.
- *Isolation:* Each agent runs in a controlled manner – potentially even isolated environment or thread if needed. Tensor0's orchestration can help manage their execution. In initial implementation, agents will likely run in-process but logically isolated by interface boundaries (each agent only interacts via its input/output, not shared globals).
- **Agent Roles vs Instances:** The workflow spec might specify an **agent role** needed for a step (e.g. "any agent with role = Translator") rather than a specific implementation. The engine can then select an appropriate agent from the pool that fulfills that role. This could allow dynamic switching – for instance, multiple LLM agents could have the role "ContentGenerator" (maybe one using GPT-4, another using a local model) and the selection could be configured globally or based on context. However, initially it's simpler to map each step to a specific agent defined in the workflow spec.
- **Lifecycle:** Agents can have a lifecycle (init, execution, teardown). For quick triggered workflows, agents likely just execute and return results. In future interactive or long-running workflows, agents might maintain memory/state. The design allows for both: stateless function-like agents now, and more stateful autonomous agents later (e.g., an agent that keeps a memory of past interactions in a multi-agent simulation).
- **Example Agent Implementation (Code Snippet):** To illustrate, here's a pseudocode for a simple LLM agent and how it could be used:

```
class LLMAgent(BaseAgent):
    def __init__(self, model_name):
        self.model_name = model_name

# (Possibly load model or set up connection; with Tensor0, just store name)
    def execute(self, inputs: dict) -> dict:
        prompt = inputs["prompt"]  # expecting the prompt text in inputs
        # Call Tensor0 to get completion (pseudo-code)
        response = tensor0.inference(model=self.model_name, prompt=prompt,
params={})
        return {"text": response.text}

# Example usage:
agent = LLMAgent(model_name="openai-gpt4")
result = agent.execute({"prompt": "Summarize the following data: ..."})
print(result["text"])
```

In real usage, the engine would handle passing the right prompt (constructed from a template and input data) to the agent. The agent focuses on its singular task (make an LLM call in this case). Similar structure would apply for other agent types (with different internal logic).

Input/Output Handling

A core requirement is that the system accept **either natural language prompts or structured JSON** to trigger workflows. This means designing a flexible input layer and a predictable output structure:

- **Natural Language Input:** When a user provides a free-form prompt (e.g., via a chat UI or command line), the system must interpret it. This could be achieved by:
- Having a special workflow that is a "router" or an interpreter (an LLM itself could classify the prompt into a workflow type).
- **OR** maintaining a simple mapping of keywords/intents to workflow specs. For instance, if the prompt contains "summarize dataset", the engine might choose the "data_summary_workflow".

Initially, a pragmatic approach is to require the user to specify which workflow they want (through the UI selection or a keyword in the prompt) if multiple workflows exist, to avoid heavy NLP parsing. However, making it user-friendly is important, so a single prompt could still be parsed by a lightweight intent classifier or even a designated agent that reads the prompt and outputs a workflow selection.

- **JSON Input:** In programmatic or automated scenarios, a JSON structure can specify everything needed to run a workflow. For example:

```
{
  "workflow": "content_generation",
  "inputs": {
     "topic": "renewable energy",
     "length": 500
  }
}
```

In this case, the engine sees the `workflow` key and loads that workflow spec. It then takes the `inputs` and maps them to the expected input names of the first step or the overall workflow (the Protobuf schema might define certain inputs at the workflow level). Structured JSON is useful for integration via APIs or if one system calls this package to do a task – it ensures the caller knows exactly what data to provide.

- **Input Validation:** Because we have a schema (Protobuf) for workflows, we can validate that required inputs are present and of the right type/format. If something is missing, the engine can throw a meaningful error (e.g., "Input 'topic' is required for workflow content_generation").

- **Outputs:** The system's outputs should be **structured and predictable**:

- If the workflow's goal is to produce a piece of content (text, image, etc.), the output might just be that content (as a string, file path, or binary data).
- If multiple results or metrics are produced, the output can be a JSON object with named fields.
- For data pipeline workflows, the output might be a reference to a data store or a summary report.

We can also have the Protobuf schema describe the expected output schema for each workflow. For example, a `WorkflowSpec` could include an `output_fields` definition. This way, consumers of the workflow (like a UI) know what to expect (similar to how an API contract works).

- **UI-driven Workflows:** In a UI context, a user might fill out a form (which under the hood produces a JSON) or click a button that corresponds to a workflow with preset parameters. The system should easily integrate with UIs: e.g., a web dashboard could list available workflows and their required inputs (which can be derived from the schema to auto-generate a form). Once triggered, the UI can show the workflow progress (taking advantage of Tensor0's real-time trace data) and display the result at the end.

- **No Real-time Interaction (Initial Phase):** It's important to note that at first, workflows are one-off executions. The user provides input, and the system runs through to completion without needing further user intervention. There is **no conversational back-and-forth mid-workflow** in the initial design. This simplifies execution (no need to pause and wait for user input). Future extensions might allow an agent to request additional input or clarification from the user mid-run, but that introduces complexity that will be tackled later.

- **Example Input/Output Flow:** For instance, a natural language input *"Generate a quarterly sales report and email it to me"* could be interpreted to select the `sales_report_email` workflow. The engine would then perhaps prompt the user (or require in JSON) additional structured info like the quarter or data source if not specified. The output of that workflow could be something like:

```
{
  "status": "success",
  "email_sent_to": "user@example.com",
  "report_filename": "Q3_report.pdf"
}
```

Meanwhile, if the user provided a JSON input with all details, the same workflow would execute and return a similar JSON confirming completion. Consistent output formatting allows the result to be easily consumed by whatever triggered the workflow (UI, another program, etc.).

Extensibility and Future Directions

While the initial design targets relatively straightforward pipelines, the architecture is built to **evolve**. Here we outline how the system can be extended or adapted to future needs:

- **Multi-Agent Systems:** In the future, we envision supporting scenarios where multiple agents run concurrently or interact in a loop (akin to simulations or autonomous agent collaborations). The current design can accommodate this by:
- Allowing steps that represent **parallel agent actions**. The Protobuf workflow schema can be extended with constructs for parallel branches or even loops. Tensor0's concurrency support would handle parallel execution under the hood.
- Introducing an **interaction protocol** between agents. For example, two agents could pass messages back and forth for a number of iterations (this could be represented as a sub-workflow or a special agent that orchestrates an internal dialogue). The modular agent design means we could create an agent that is essentially a manager of other sub-agents (like a facilitator).

- Retaining determinism by specifying in the workflow how agents are spawned and how many iterations or conditions for their interaction. This ensures even multi-agent runs are reproducible (given the same random seeds for the LLMs, etc.).

- **Deep Research Workflows:** Deep research often involves iterative exploration (e.g., searching for information, refining hypotheses, asking multiple questions). Our system could support this by:

- Allowing an agent to perform a **search or tool use** repeatedly until certain criteria are met. This could be an extension of the agent interface: e.g., an agent that has an internal loop (like "ResearchAgent" that keeps querying a knowledge base until it finds an answer).
- Integrating memory or knowledge base components. We might add a **Memory agent** or a shared memory context that agents can read/write to (like a blackboard system). The Protobuf spec could define which agents have access to a shared memory, enabling complex reasoning over multiple steps.

- Chaining workflows: One workflow's output might trigger another workflow. The engine could handle this by allowing workflows to be invoked as steps within a larger workflow (recursion or hierarchy of workflows). This is easily described in a schema by having a step action like `RunWorkflow(name=...)`.

- **TinyTroupe Inspiration:** TinyTroupe demonstrates rich multi-persona simulations within a structured environment. We draw inspiration from its fluid structure:

- **Persona and Environment Models:** In our future design, we can incorporate the idea of agent personas or profiles (similar to TinyTroupe's `TinyPerson`). Our agent specs could be extended to include persona traits or long-term goals, which would be more relevant when agents interact over time. For example, an agent role "ResearcherAgent" could have a persona that biases how it approaches tasks (cautious vs. aggressive research strategies).
- **Scenario Scripts:** TinyTroupe allows defining scenarios programmatically (even via JSON) for flexible simulation [9]. Similarly, our workflows could become more **data-driven scripts**. Advanced users might write a high-level script (in JSON or a domain-specific language) that we compile into a WorkflowSpec. This would make creating new complex workflows easier without coding each agent behavior from scratch.

- **Fluid Interactions:** The term "fluidity" implies the system can handle non-linear interactions smoothly (e.g., interruptions, dynamic re-planning). Achieving this might involve adding a **Dynamic Planner Agent** that can on-the-fly modify the workflow (insert or skip steps) based on runtime info. Thanks to the orchestrator's flexibility, we could in future let an agent adjust the remaining workflow by returning a decision (for instance, an agent decides more data is needed and spawns a new step to fetch it). The deterministic nature can still be preserved by constraining such behaviors through the spec (i.e., pre-defining that certain decision points allow branching).

- **Modular Extensions:** The package will be designed as an open framework where new components can plug in:

- New **Agent types** can be added by implementing the base interface and registering them. For example, adding a `VisualizationAgent` that takes data and produces a chart image.
- Support for additional **orchestration backends**: While Tensor0 is the default (and preferred for performance), the system could be adapted to run on other backends (e.g., a local task queue)

for offline or simpler use cases. The engine's calls to Tensor0 could be abstracted behind an interface so that swapping the orchestrator is possible.

- **Configuration and Policy:** Using Protobuf and a config-driven approach allows easy extension of configuration. For instance, we can introduce new fields like `timeout` or `retry_policy` for steps without breaking older workflows (they would just use defaults). As the needs arise (say we start doing long knowledge graph building tasks), we can tune these settings per workflow.
- **Integration with Other Systems:** The structured I/O and modular design mean we can integrate with external systems. For example:
  - Expose each workflow as a **microservice endpoint** (the engine can be hosted behind a REST API, where each workflow is an endpoint, leveraging the known input/output schema).
  - Log data to external monitoring or BI systems (since Tensor0 already logs to ClickHouse by default [2], we can tap into that or forward events to other dashboards).
  - Use a **schema registry** (like Buf or Confluent Schema Registry) for versioning workflow definitions if many services/teams define their workflows. This ensures updates are managed in a collaborative way.

In summary, the system is built with change in mind. The clean separation of concerns (input handling, workflow spec, orchestration, agent implementations) means we can evolve each part independently. The use of strong schemas and an orchestration backbone provides confidence that as we extend into multi-agent and more complex domains, we won't hit a performance or maintainability wall. The design principles echo the flexibility seen in TinyTroupe's approach to defining agents and scenarios programmatically [3], while focusing on our core initial use cases.

Input/Output Design Details

(This section delves a bit more into how inputs and outputs are structured and how a developer or user would work with the system.)

- **Defining Workflow Inputs:** Each workflow can define what inputs it expects. For example, a content generation workflow might expect an input field `topic` (string) and `length` (integer for word count). In the Protobuf schema, we might extend `WorkflowSpec` to include an `InputSpec` (essentially a list of expected input fields with types). This allows automatic validation and also UI generation. A developer defining a workflow will specify these in the .proto file or via a builder API.
- **Natural Language to Structured Input:** One interesting feature could be the ability to parse a natural language prompt into structured inputs using the LLM itself. For example, if a user says: *"Summarize the attached CSV file and plot the trends."*, the system could use a preliminary LLM call to interpret this: identify that this corresponds to the "data_analysis" workflow, and extract that the file `attached CSV` is the target data, and that an output plot is requested. This is akin to a **prompt parser agent**. While this is a possible extension, initially we will likely rely on simpler methods (like explicit user instructions or UI selection) to map prompts to workflows.
- **Example JSON Input Format:** To illustrate, suppose we have a workflow named `"data_pipeline_example"` that ingests a dataset, processes it, and returns stats. A JSON input could look like:

```
{
  "workflow": "data_pipeline_example",
  "inputs": {
    "data_source": "s3://mybucket/data.csv",
```

```
    "filter_criteria": "country = US"
  }
}
```

The engine will:

- Locate `data_pipeline_example` in the workflow registry.
- Parse and validate that `data_source` and `filter_criteria` are provided and of correct type.
- Possibly convert `data_source` into an internal format (if needed).
- Execute the workflow. Each step will pull what it needs from this `inputs` pool or from prior steps.
- **Output Structures:** We aim for outputs that are easy to consume. If a workflow has multiple outputs, these can be returned as fields in a JSON. For instance:

```
{
  "summary_text": "...",
  "record_count": 1500,
  "errors": []
}
```

In a content generation use case, maybe the output is just `"content": "<generated article text>"`. Consistency with the schema is key – ideally, the schema defines the output format too (similar to how function return types are defined).

- **Error Reporting:** Outputs should also convey errors in a structured way. Instead of just failing silently or throwing exceptions, the system can return an output like:

```
{
  "status": "failed",
  "error": "API X returned 500 error",
  "step": "fetch_data"
}
```

This way, the calling context (user or program) knows what happened. Internally, Tensor0 and our engine will catch exceptions or error statuses and format them into such error outputs.

- **UI/UX Considerations:** If a user is interacting via a UI, the system can provide **progress updates**. For example, after each step, a short message could be logged (via websockets or similar) like "Step 1/3 completed: Data fetched (1000 records)". This is possible since Tensor0 can give asynchronous feedback or we can instrument the engine to emit events after each step. This greatly improves user experience for longer workflows.
- **Security and Validation:** Accepting prompts and executing arbitrary workflows raises security considerations:
- We will ensure that **structured JSON inputs are validated** strictly against the workflow schema to avoid injection or misuse (e.g., if a prompt template expects a string, ensure no malicious content can break out of the prompt format).
- Agents that call external services (APIs, tools) should sanitize inputs (for instance, if an agent runs a shell command or database query, guard against injection).
- In a multi-tenant setting or if exposing this system as a service, we might need authentication and permission checks on which workflows can be run and by whom. This is outside the core design but something to be aware of.

- **Internationalization of Prompts:** If natural language prompts are used, consider that the system could be extended to handle multiple languages (for global users) by either translating inputs or having multilingual prompt parsing. This again hints that having a structured layer (the JSON) is easier to standardize across languages, whereas direct prompt understanding might require language-specific handling or using an LLM.

Sample Use Cases

To ground the design in concrete scenarios, here are a few sample use cases and how the system would handle them:

1. Data Pipeline Automation

Use Case: A data engineer wants to automate a pipeline that takes a raw dataset, cleans it, analyzes it, and generates a report.

- **Workflow:** `"data_cleaning_analysis"` – defined to have steps:
- **Load Data (Agent: DataLoader)** – reads a file from a given path or URL.
- **Clean Data (Agent: DataCleaner)** – applies transformations (e.g., remove nulls, filter by criteria).
- **Analyze Data (Agent: DataAnalyzer)** – perhaps runs some summary statistics or ML model.
- **Report Generation (Agent: ReportGenerator)** – uses an LLM to draft a summary of the findings, maybe even generating graphs or tables if needed.
- **Input:** Could be triggered via JSON:

```
{
  "workflow": "data_cleaning_analysis",
  "inputs": { "data_path": "datasets/sales_q4.csv", "region": "EU" }
}
```

  The workflow spec knows `data_path` goes to step1 input, and `region` is used by DataCleaner to filter data.
- **Output:** Possibly a JSON with the report location or content:

```
{
  "status": "success",
  "rows_processed": 10500,
  "report_file": "reports/sales_q4_summary.pdf"
}
```

- **Behind the scenes:** Tensor0 orchestrates the first three steps potentially in sequence (though if analysis can run in parallel with some cleaning processes, that's possible). The LLM in step 4 might be a heavy call (GPT-4) but Tensor0 will handle it efficiently, and its result (the report text) can be combined with embedded charts if the ReportGenerator agent was sophisticated (maybe it got charts from DataAnalyzer and inserted them into a template).

This use case shows how **data pipelines** benefit from a mix of traditional processing and LLM generation, all within one unified workflow.

2. Content Generation and Publishing

Use Case: A marketing team wants to generate content (like blog posts or social media posts) given an outline or topic, and then automatically publish it via an API.

- **Workflow:** `"blog_post_generation"` – steps could be:
- **Outline Expansion (Agent: LLMPlanner)** – take a short prompt or outline and use an LLM to expand it into a full outline or first draft.
- **Content Refinement (Agent: LLMRefiner)** – perhaps another LLM call or a series of calls to improve the draft (fix grammar, expand sections). Alternatively, this step could use a different model or a series of prompts for each section of the outline (the workflow could loop here, though initial version might do it in one go).
- **SEO Optimization (Agent: SEOAnalyzer)** – analyzes the text and possibly calls an external SEO API to get keyword suggestions, then adjusts the text (this could be another LLM prompt: "insert these keywords naturally").
- **Publish (Agent: CMSUploader)** – calls a content management system API to create a new post with the final content.
- **Input:** Possibly just a prompt like:

  ```
  "Write a blog post about the benefits of renewable energy."
  ```

  If provided via JSON, it might include a title or desired word count:

  ```
  {
    "workflow": "blog_post_generation",
    "inputs": { "title": "Benefits of Renewable Energy", "length": 800 }
  }
  ```

- **Output:** Could be a confirmation with the published post URL:

  ```
  {
    "status": "success",
    "url": "https://myblog.com/benefits-of-renewable-energy"
  }
  ```

  And perhaps the text of the post for record.
- **Highlights:** This use case chains multiple LLM calls (steps 1 and 2, possibly 3) with an external API call (step 4). The system demonstrates **API chaining** – output of one step (the content) becomes input to the publishing API. It also shows how different agents can collaborate: an LLM focused on content, a tool focused on analysis, etc. Because Tensor0 handles episodes, we could treat the whole content creation as one episode of prompts, which might allow capturing the iterative refinement process for evaluation.

3. Multi-API Orchestration (Customer Support Example)

Use Case: A customer support workflow where a user's query needs to go through multiple systems: first fetch info from a knowledge base, then maybe summarize it, then create a ticket in a system.

- **Workflow:** `"answer_and_ticket"` – steps:

- **Search KB (Agent: KnowledgeSearcher)** – takes a query and calls a KB API or database to find relevant articles/answers.
- **Compose Answer (Agent: QnAComposer)** – uses an LLM to draft a personalized answer to the user's question based on the KB results (the agent might feed the found info into a prompt).
- **Create Ticket (Agent: TicketCreator)** – if needed, creates a follow-up ticket in an issue tracking system via API (e.g., if the answer requires engineering intervention or logging the incident).
- **Input:** Natural language prompt example: "User asks: 'How do I reset my password?' – provide answer and log a support ticket." In JSON, that might be:

```
{
  "workflow": "answer_and_ticket",
  "inputs": { "user_query": "How do I reset my password?", "user_id":
"12345" }
}
```

- **Output:** Perhaps:

```
{
  "answer": "Hi, to reset your password, you can ...",
  "ticket_id": "SUP-009876"
}
```

  The answer would also be delivered to the user (via the chat or email, outside the workflow scope).
- **Note:** This is triggered by an event (user question) and runs through. It's not an interactive chat, but it achieves a multi-step resolution. It shows how this system can act like a flexible **AI-powered middleware**: integrating knowledge retrieval, LLM Q&A, and enterprise system actions. All steps are deterministic in sequence, but the content of the answer is generated. The use of Tensor0 ensures the LLM call is efficient and logged. If the KB returns no result, the workflow could have conditional logic (e.g., skip to step 3 to create a ticket for a human to follow up, with a different message). Such conditional branching can be encoded in the workflow spec (future extension) or handled by the agent (the QnAComposer could decide "no answer found, escalate").

4. Extensible Multi-Agent Research (Future Scenario)

(An example to illustrate how the design might handle a more complex scenario down the line.)

Use Case: A scholar uses the system to conduct a literature review. They want multiple agents to collaboratively find and summarize information on a topic.

- **Workflow:** `"literature_review_multi_agent"` – possible structure:
- Agents: define a pool of agents like `ResearcherAgent`, `CriticAgent`, `WriterAgent`. The **ResearcherAgent** searches for papers (via an API or web), the **CriticAgent** evaluates the quality or relevance of found info, and the **WriterAgent** compiles the summary.
- The workflow might orchestrate a loop:
  1. ResearcherAgent finds some sources (step produces a list of papers).
  2. CriticAgent reviews and filters sources (produces a shortlist).
  3. If more info needed, loop back to ResearcherAgent with refined criteria (this could be decided by CriticAgent's output).

4. Once enough info, WriterAgent generates a summary of all findings.

- This involves **dynamic interaction**: effectively a mini multi-agent system. Our design would handle this by either explicitly coding the loop in the workflow spec or by using an agent to control it. A simplistic way in current design is to allow a step to repeat (a feature to add in schema), or treat the whole cycle as an agent's internal logic (e.g., a `LiteratureReviewAgent` that internally uses sub-agents – not ideal, better to have it in the workflow for transparency).
- **Input:** Could be a prompt like "Research the impacts of climate change on agriculture and summarize key findings."
- **Output:** A detailed report, maybe with sections, citations, etc., produced by the WriterAgent.
- **This scenario** showcases where we want to head: the ability to coordinate multiple specialized agents towards a complex goal. The current architecture can evolve to support it, mainly by enhancing the workflow schema and engine's logic for loops/conditions, and by possibly introducing a **planner agent** that can modify the course of the workflow on the fly. By laying the groundwork with deterministic, modular steps now, we will be able to incrementally add such sophistication.

Conclusion

This design document has outlined a modular prompt-to-workflow system that bridges the gap between natural language instructions and rigorous, automated workflows. By combining **structured workflow definitions (Protobuf)** with a powerful **execution orchestrator (Tensor0)** and a library of modular **agents**, the system achieves both **flexibility** and **reliability**.

In summary, the design emphasizes:

- **Clear structure** (using schemas and defined components) alongside **fluid execution** (multi-step orchestration with an LLM/tool backend), much like TinyTroupe achieves fluid multi-agent simulations on a structured backbone [9].
- Initial focus on practical workflows (data pipelines, content generation, API orchestration) with correct, observable, and efficient execution.
- A forward-looking architecture that can grow into supporting advanced AI agent interactions and research-oriented workflows, ensuring that today's design decisions won't constrain tomorrow's capabilities.

With this foundation, developers and domain experts can create sophisticated AI-infused workflows by writing high-level specifications and letting the system handle the heavy lifting of execution. Whether it's automating business processes or enabling new forms of AI-driven research, this prompt-to-workflow system aims to provide a robust canvas for innovation.

---

[1] [2] [4] [5] [6] [7] [8] Overview - TensorZero Docs
https://www.tensorzero.com/docs/gateway/index

[3] [9] Introducing TinyTroupe: Microsoft's New AI Agents | by Devashish Datt Mamgain | Python in Plain English
https://python.plainenglish.io/introducing-tinytroupe-microsofts-new-ai-agents-05e4b1ba4eb5?gi=c0828bac1efa